# Final Exam

EE 3223: C++ and Data Structures — UTSA Summer 2026

**Covers:** All 20 lectures (L1–L20). Weighted toward Part III (operator overloading, inheri-tance/polymorphism, templates/STL, BSTs, hashing).

**Submission:** Upload a single PDF with your answers for Parts A–C, plus your `.cpp` file for Part D, to Canvas before the window closes.

**Academic integrity:** You may use your notes, the lecture slides, and the lecture code files. You may not receive help from any person or use AI tools.

---

## Part A — Multiple Choice (20 pts)

Circle the letter of the best answer. (4 pts each)

**1.** A pure virtual function in a base class:

- ○ (a) Must have a body defined in the base class
- ◉ (b) Makes the class abstract — it cannot be instantiated directly
- ○ (c) Is automatically overridden by derived class constructors
- ○ (d) Can only be a destructor

---

**2.** The primary benefit of `template <typename T>` for a function is:

- ○ (a) It runs faster than a non-templated version at runtime
- ◉ (b) It generates type-specific code at compile time with zero runtime overhead
- ○ (c) It allows the function to be called with any number of arguments
- ○ (d) It requires the programmer to specify types explicitly at each call site

---

**3.** Which BST traversal visits nodes in ascending sorted order?

○ (a) Preorder (root → left → right)

○ (b) Postorder (left → right → root)

◉ (c) Inorder (left → root → right)

○ (d) Level-order (breadth-first)

---

**4.** For a hash table using separate chaining, average O(1) search requires:

○ (a) A perfect hash function with no collisions

◉ (b) A small load factor  (average chain length  1)

○ (c) Each bucket to be a sorted array

○ (d) At least as many buckets as total keys ever inserted

---

**5.** When deleting a BST node with two children, the standard approach is:

○ (a) Promote the left child to the deleted node's position

◉ (b) Find the inorder successor (leftmost node of the right subtree), copy its value into the deleted node, then recursively delete the successor

○ (c) Rebalance the entire tree

○ (d) Split into two separate BSTs

---

## Part B — Code Trace (30 pts)

Write the **exact output** each program produces. (10 pts each)

**6.**

```
#include <iostream>
using namespace std;
template <typename T>
T mymax(T a, T b) { return (a > b) ? a : b; }
int main() {
    cout << mymax(3, 7) << "\n";
    cout << mymax(2.5, 1.8) << "\n";
    cout << mymax('z', 'a') << "\n";
    return 0;
}
```

Output:
```
7
2.5
z
```

**7.**

```cpp
#include <iostream>
using namespace std;
class Animal {
public:
    virtual void speak() const { cout << "...\n"; }
};
class Dog : public Animal {
public:
    void speak() const override { cout << "Woof\n"; }
};
class Cat : public Animal {
public:
    void speak() const override { cout << "Meow\n"; }
};
int main() {
    Animal* pets[] = { new Dog(), new Cat(), new Dog() };
    for (int i = 0; i < 3; i++) { pets[i]->speak(); }
    return 0;
}
```

Output:
```
Woof
Meow
Woof
```

**8.**

```cpp
#include <iostream>
using namespace std;
struct Node { int val; Node* left; Node* right; };
Node* insert(Node* root, int val) {
    if (!root) { return new Node{val, nullptr, nullptr}; }
    if (val < root->val) { root->left  = insert(root->left,  val); }
    else if (val > root->val) { root->right = insert(root->right, val); }
    return root;
}
void inorder(Node* root) {
    if (!root) { return; }
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}
int main() {
    Node* t = nullptr;
    for (int v : {5, 2, 8, 1, 4}) { t = insert(t, v); }
    inorder(t);
```

```
    cout << "\n";
    return 0;
}
```

Output: 1 2 4 5 8

---

## Part C — Short Answer (20 pts)

Answer in 2–4 sentences. (5 pts each)

**9.** What does `virtual ~Base() {}` in a base class do, and what happens if you omit it when you call `delete` on a derived object through a base pointer?

**10.** Explain the difference between a function template and a class template. When would you use a class template instead of a function template?

**11.** In linear probing, what is clustering? Why does increasing the load factor ( ) make clustering worse and degrade performance?

**12.** Insert the values {10, 5, 15, 3, 7} into an empty BST in that order. What does the inorder traversal produce? Which node becomes the root?

---

## Part D — Programming (30 pts)

**File:** `partD.cpp`

The file must compile with: `g++ -std=c++17 -Wall -Wextra -o partD partD.cpp`

---

Implement an `IntStack` class backed by a heap-allocated array (not a linked list). The constructor takes the stack capacity. The class must provide:

- `IntStack(int capacity)` — allocates array of given capacity.
- `~IntStack()` — frees the array.
- `bool empty() const`
- `bool full() const`
- `void push(int v)` — push; do nothing if full.
- `void pop()` — pop; do nothing if empty.
- `int top() const` — return top (assume not empty).
- `void print() const` — print top to bottom, space-separated, newline at end.

In `main()`, create an `IntStack` of capacity 5, push {10, 20, 30}, print, print the top, pop, push 99, and print again.

Expected output:

```
30 20 10
top: 30
99 20 10
```

Starter code:

```
#include <iostream>
using namespace std;

class IntStack {
    int* data_;
    int  cap_;
    int  sz_;
public:
    IntStack(int capacity);
    ~IntStack();
    bool empty() const;
    bool full()  const;
    void push(int v);
    void pop();
    int  top() const;
    void print() const;
};

int main() {
    IntStack s(5);
    s.push(10); s.push(20); s.push(30);
    s.print();
    cout << "top: " << s.top() << "\n";
    s.pop();
    s.push(99);
    s.print();
    return 0;
}
```

## Part C — Short Answer (Answer Sheet)

*Type your response in the box below each question. Answer in 2–4 sentences (5 pts each).*

**9. What does virtual ~Base() {} do, and what happens if you omit it when calling delete on a derived object through a base pointer?**

It defines a virtual base destructor for the base constructor. The programmer must make a derived destructor and when called it will activate both. If you do not have it when calling delete on a derived object through a base pointer then you will get undefined behavior. This could be something like memory leaks.

**10. Explain the difference between a function template and a class template. When would you use a class template instead of a function template?**

A function template will execute while keeping its variable type only for the duration of its use. The class will keep its function type across multiple calls as the function continues to get used. If you need to use the created item multiple times or for various different uses that would be in a class, you would use the class template. If you just need to make a specific variable version of a function and don't intend to use it again, you will be fine with the template function.

**11. In linear probing, what is clustering? Why does increasing the load factor make clustering worse and degrade performance?**

Clustering is when a large amount of slots get occupied next to one another in a hash table. When the load factor increases, these clusters increase in size, causing the program to have to search for longer to find an unoccupied slot. These longer search times lead to worse performance overall.

**12. Insert {10, 5, 15, 3, 7} into an empty BST in that order. What does the inorder traversal produce? Which node becomes the root?**

The in order traversal produces: 3, 5, 7, 10, 15
The root will be 7 since it's in the middle. From there, 5 will be to its left and 10 will be to its right. 3 will be to the left of 5 and nullptr to the right, while nullptr is left of 10 and 15 to the right.